Rapport — Gestion des livraisons

TP d'algorithmique : gestion des colis et optimisation du chargement

Langage retenu : Python 3.10+. Python rend les structures et les algorithmes faciles à lire, tester et expliquer. La solution n'utilise aucune bibliothèque externe.

1. Structures de données et gestion

Les colis sont stockés dans un dictionnaire indexé par identifiant. La recherche, l'ajout et la détection d'un doublon coûtent O(1) en moyenne, avec un coût défavorable pouvant atteindre O(n) dans des cas de collisions. La mémoire occupée est O(n). Ce choix convient à une collection importante lorsque les recherches par identifiant sont fréquentes. Une liste seule imposerait une recherche linéaire O(n).

2. Tri par poids, valeur ou priorité

Le programme construit une liste des colis puis la trie avec le tri intégré de Python (Timsort). La complexité usuelle et dans le pire cas est O(n log n), avec O(n) de mémoire supplémentaire dans cette utilisation. C'est adapté à 100 000 éléments pour des tris ponctuels. Si les mêmes tris sont demandés très souvent sans modification des colis, on peut conserver des index triés et les mettre à jour lors des ajouts, au prix d'une gestion plus complexe.

3. Préparation du camion : programmation dynamique

Le problème est un sac à dos 0/1 : chaque colis est pris au plus une fois, le poids total ne dépasse pas la capacité C et le score total est maximisé. Pour chaque capacité de 0 à C, dp[c] conserve le meilleur score connu. Les capacités sont parcourues en ordre décroissant pour empêcher qu'un colis soit réutilisé dans la même étape. Une matrice compacte de décisions permet de reconstruire le chargement retenu.

Complexité : O(nC) en temps et O(C+nC) en mémoire avec la reconstruction. Cette méthode est exacte pour le score choisi, mais pseudo-polynomiale : le coût dépend de la valeur numérique de C, et pas seulement de la taille de l'entrée. Elle est adaptée à une capacité entière modérée. Avec une capacité immense, il faudrait envisager une approximation, un algorithme de branchement et bornes, ou une autre formulation.

4. Règle de priorité

Le score proposé est valeur × (priorité + 3), avec une priorité de 1 à 5. Ainsi, un colis de priorité 5 reçoit un multiplicateur de 8 et un colis de priorité 1 un multiplicateur de 4. La règle est simple et explicite, mais elle modifie l'objectif : la solution maximise le score pondéré et non nécessairement la valeur brute. Si l'entreprise exige la valeur monétaire maximale avant tout, il faut optimiser la valeur et appliquer la priorité comme départage ou comme contrainte métier.

5. Mesures et passage à l'échelle

Le menu propose un benchmark pour 100, 1 000, 10 000 et 100 000 colis. Il mesure l'ajout, le tri et la recherche. Les temps doivent être exécutés sur la machine de test et reportés dans le compte rendu : ils varient selon le matériel et la charge. L'ajout de n éléments dans le dictionnaire est O(n) au total en moyenne ; un tri est O(n log n) ; une recherche individuelle est O(1) en moyenne. Le calcul du chargement est le point sensible si n ou C devient très grand.

6. Autres choix possibles

Élément	Solution	Complexité	Alternative
Recherche ID	Dictionnaire	O(1) moyen	Liste : O(n)
Organisation	Tri Timsort	O(n log n)	Index triés maintenus
Chargement	Programmation dynamique	O(nC)	Glouton (rapide mais non optimal), branchement et bornes

7. Réponses aux questions finales

1. Choix le plus important : la programmation dynamique, car elle fournit une solution optimale pour le score défini, contrairement à une stratégie gloutonne générale.

2. Partie problématique avec des millions de colis : le chargement en O(nC), surtout si la capacité est élevée ; les tris complets répétés peuvent aussi coûter cher.

3. Vitesse ou mémoire : le dictionnaire consomme de la mémoire pour accélérer les recherches. Pour le sac à dos, dp est compressé en O(C), mais la reconstruction nécessite O(nC) bits de décisions.

4. Autre stratégie : un glouton par score/poids serait rapide, mais ne garantit pas l'optimum. Un algorithme approché pourrait mieux passer à l'échelle.

5. Avec C : les algorithmes resteraient similaires, mais il faudrait gérer manuellement les structures et allocations mémoire. C permet plus de contrôle et potentiellement moins de surcoût, au prix d'un code plus long et d'un risque accru d'erreurs.

À compléter avant le rendu : ajouter les temps mesurés réellement sur votre machine et joindre l'URL du dépôt GitHub si elle est demandée.